Dhirubhai Ambani University
High Performance Computing (HPC)
Assignment 05 – Parallel Insertion and Deletion in Mover

Radhika Sanagadhiya (202301184)
Hiya Modi (202301011)

Instructor: Dr. Bhaskar Chaudhury
Teaching Assistants: Ayushi Sharma, Kaushik Prajapati, Libin Varghese
Date: March 31, 2026

1 Introduction

This assignment builds upon Assignments 03 and 04 by incorporating dynamic particle management in the **Mover** operation. Particles move randomly within the 1×1 domain using displacements $r_x \in [-\Delta x, \Delta x]$ and $r_y \in [-\Delta y, \Delta y]$, where $\Delta x = 1/N_x$ and $\Delta y = 1/N_y$. If a particle exits the domain ($x, y \notin [0, 1]$), it is deleted, and an equal number of new particles are inserted at random positions inside the domain to keep the total particle count constant.

Two implementation approaches were developed and compared:

1. **Deferred Insertion Approach**: All particles are moved first; deletions are counted and voids are pushed to the end; new particles are inserted afterward.

2. **Immediate Replacement Approach**: If a particle exits, it is immediately replaced at the same memory location with a new random particle.

All computations use `double` for floating-point values. The Mover was parallelized using OpenMP. Experiments were performed on both Lab PC and HPC Cluster.

2 Particle Distribution Verification

To ensure the random number generation and insertion logic do not introduce bias or clustering, particle positions were verified.

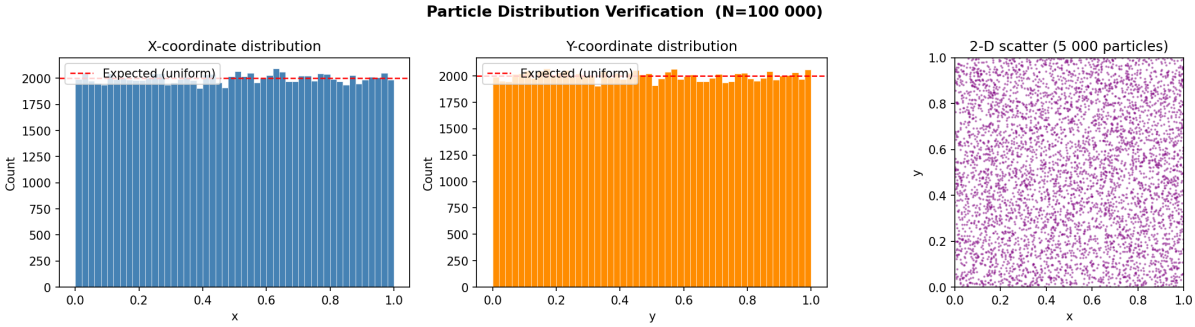

Figure 1: Particle Distribution Verification (N=100,000). Histograms of X and Y coordinates closely match the expected uniform distribution (red dashed line). The 2D scatter plot shows uniform coverage without artificial clustering.

The histograms and scatter plot confirm uniform distribution across the domain, validating the correctness of the insertion and random number generation logic.

3 Experiment 01: Scaling of Execution Time with Particle Count

3.1 Execution Time Scaling

Tests were conducted for three grid configurations with particle counts ranging from 10^2 to 10^8 (Maxiter = 10). Total time is the sum of Interpolation + Mover timings over all iterations.

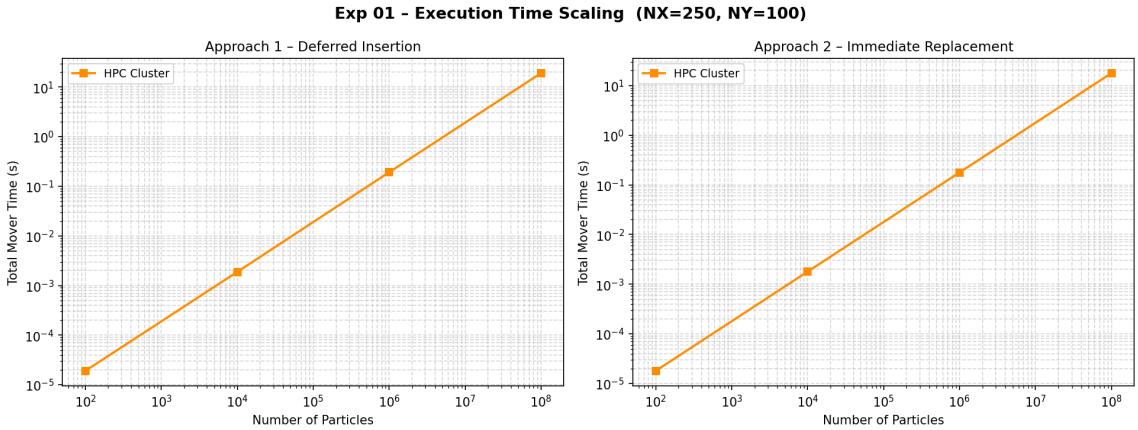

Figure 2: Execution Time Scaling (NX=250, NY=100) on HPC Cluster – Both approaches show near-linear scaling (log-log plot).

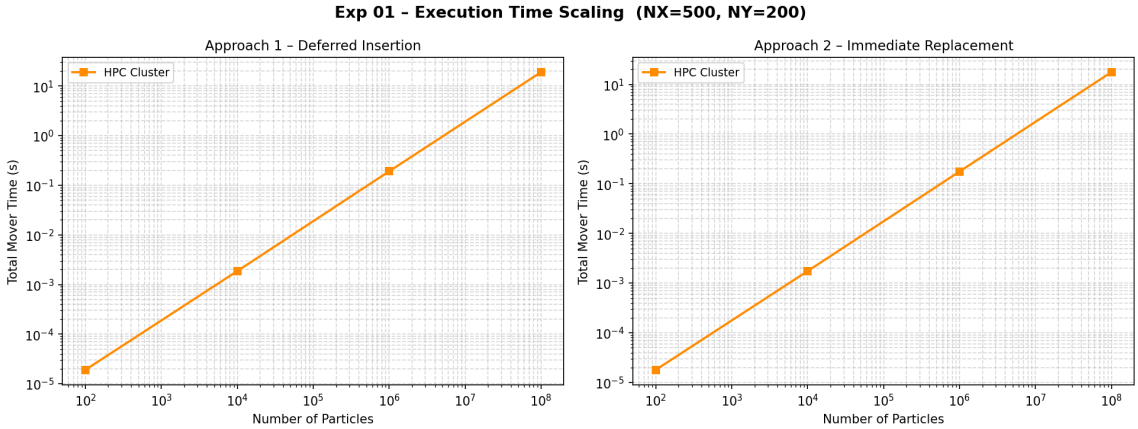

Figure 3: Execution Time Scaling (NX=500, NY=200) on HPC Cluster.

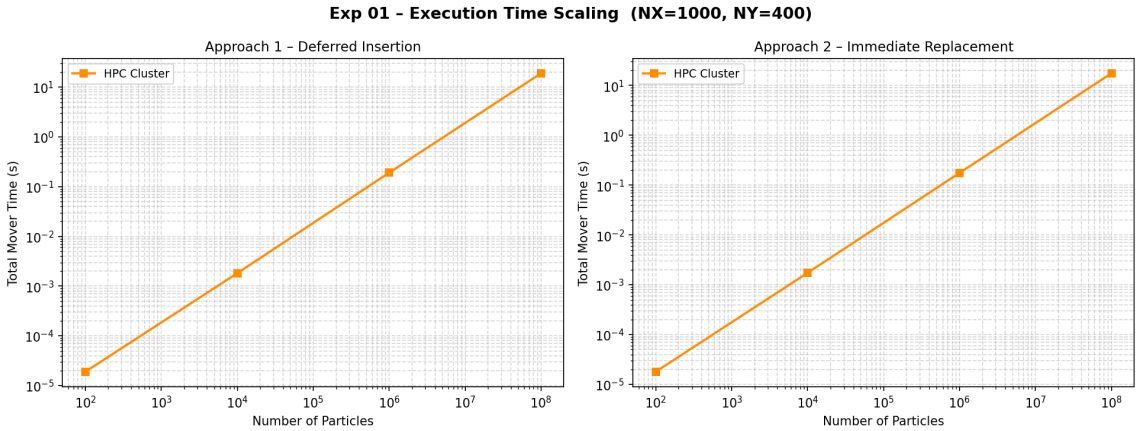

Figure 4: Execution Time Scaling (NX=1000, NY=400) on HPC Cluster.

Observation: Both approaches exhibit approximately linear scaling ($O(N)$) with the number of particles, as expected for particle-centric operations. Performance is very similar between Deferred and Immediate approaches, with minor variations at large particle counts due to memory access patterns.

3.2 Per-Particle Execution Time vs Particles Per Cell (PPC)

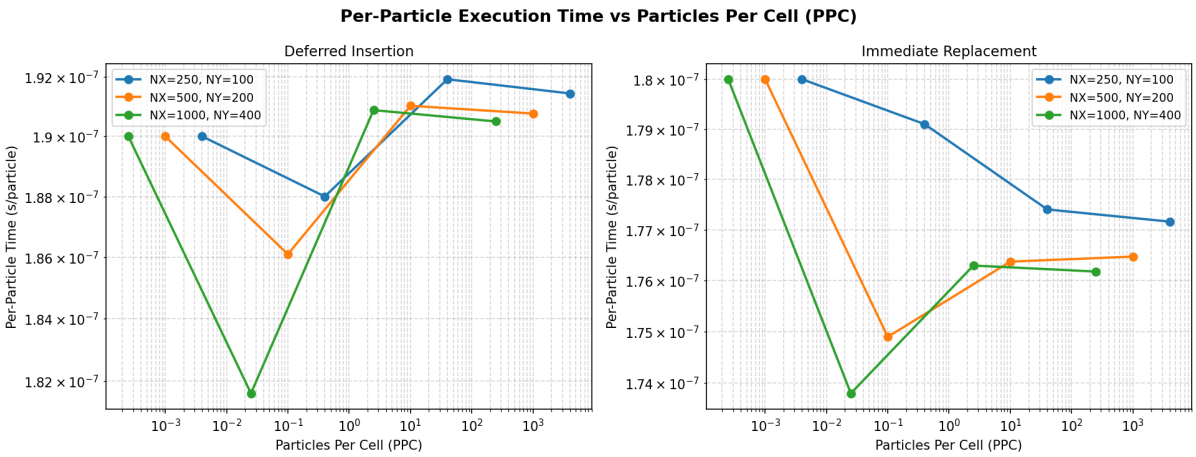

Figure 5: Per-Particle Execution Time vs Particles Per Cell (PPC) for Deferred Insertion (left) and Immediate Replacement (right).

Analysis: The per-particle execution time remains remarkably stable ($\approx 1.82 \times 10^{-7}$ to 1.92×10^{-7} s/particle) across a wide range of PPC (from 10^{-3} to 10^{3}). Minor fluctuations exist, but there is no strong trend with PPC. This indicates that the Mover + insertion/deletion operations are primarily **compute-bound** per particle rather than heavily affected by cache locality changes due to particle density. Random number generation and boundary checks dominate the cost consistently.

4 Experiment 02: Parallel Implementation and Scalability

The Mover operation was parallelized using OpenMP. Experiments used ≈14 million particles and Maxiter = 10, with 2, 4, 8, and 16 threads. Results are compared with Assignment 04 (no insertion/deletion).

4.1 Runtime vs Number of Threads

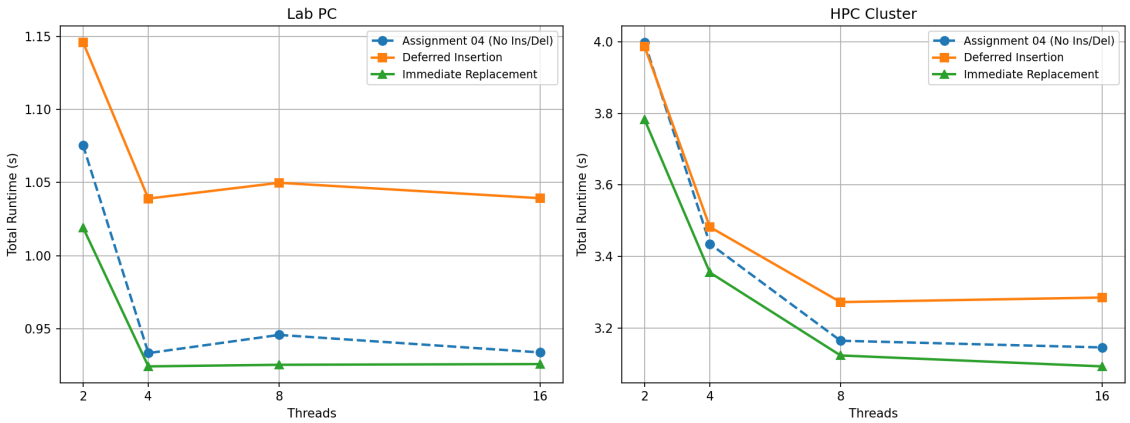

Figure 6: Total Runtime vs Threads (NX=250, NY=100) on Lab PC (left) and HPC Cluster (right).

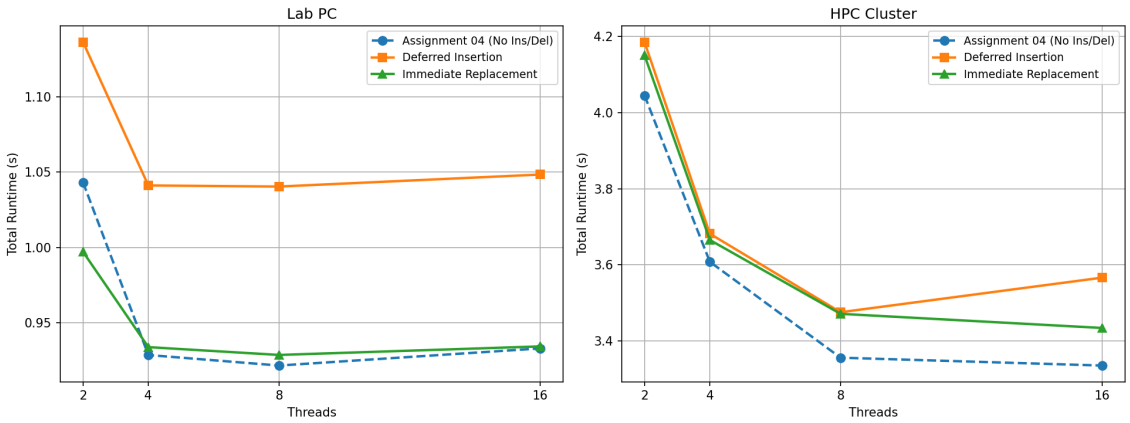

Figure 7: Total Runtime vs Threads (NX=500, NY=200) on Lab PC (left) and HPC Cluster (right).

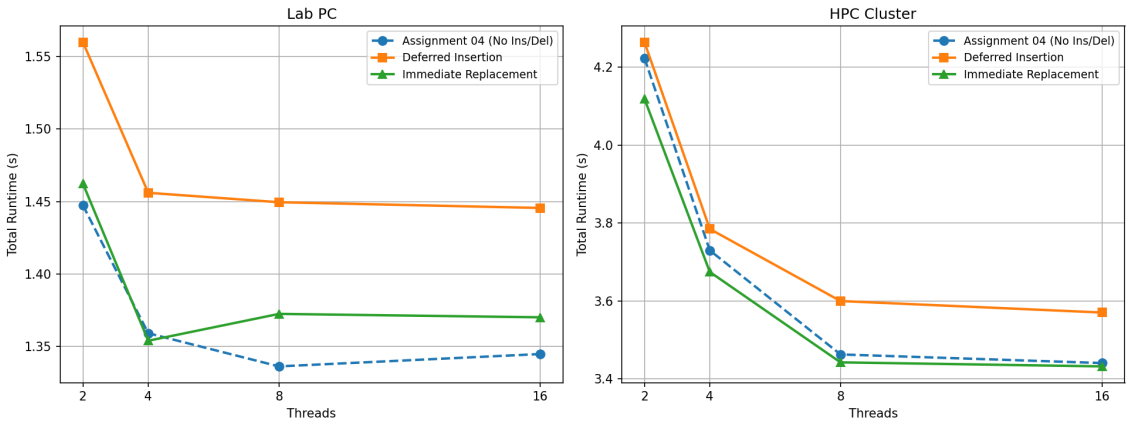

Figure 8: Total Runtime vs Threads (NX=1000, NY=400) on Lab PC (left) and HPC Cluster (right).

4.2 Speedup Analysis

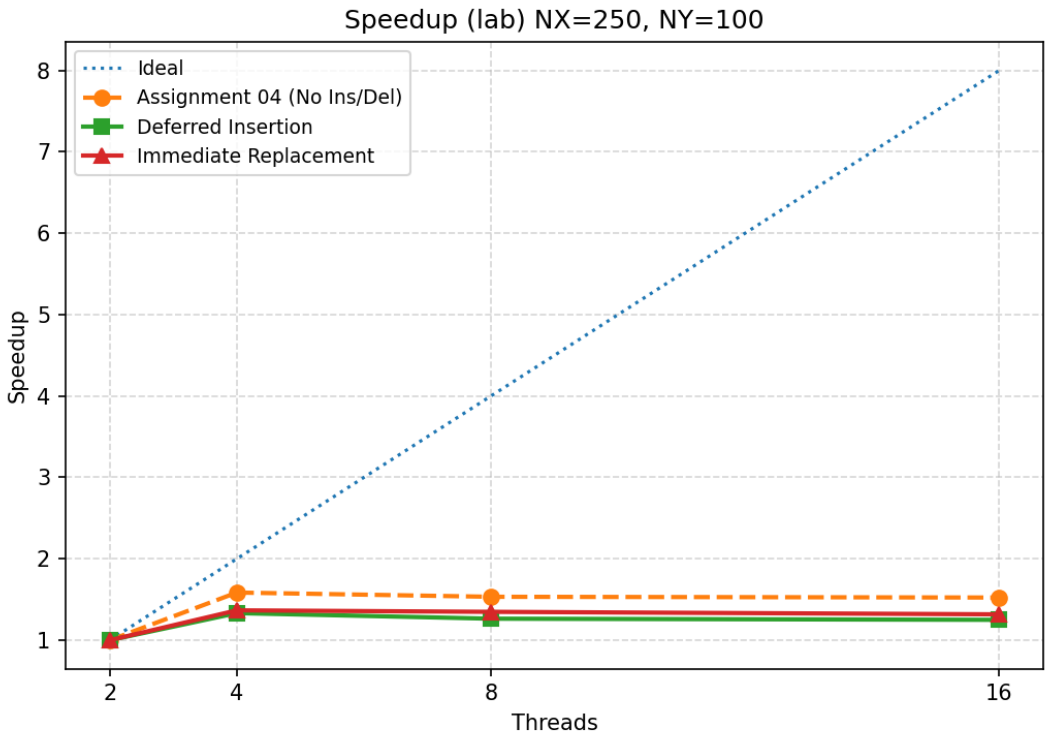

Figure 9: Speedup on Lab PC (NX=250, NY=100).

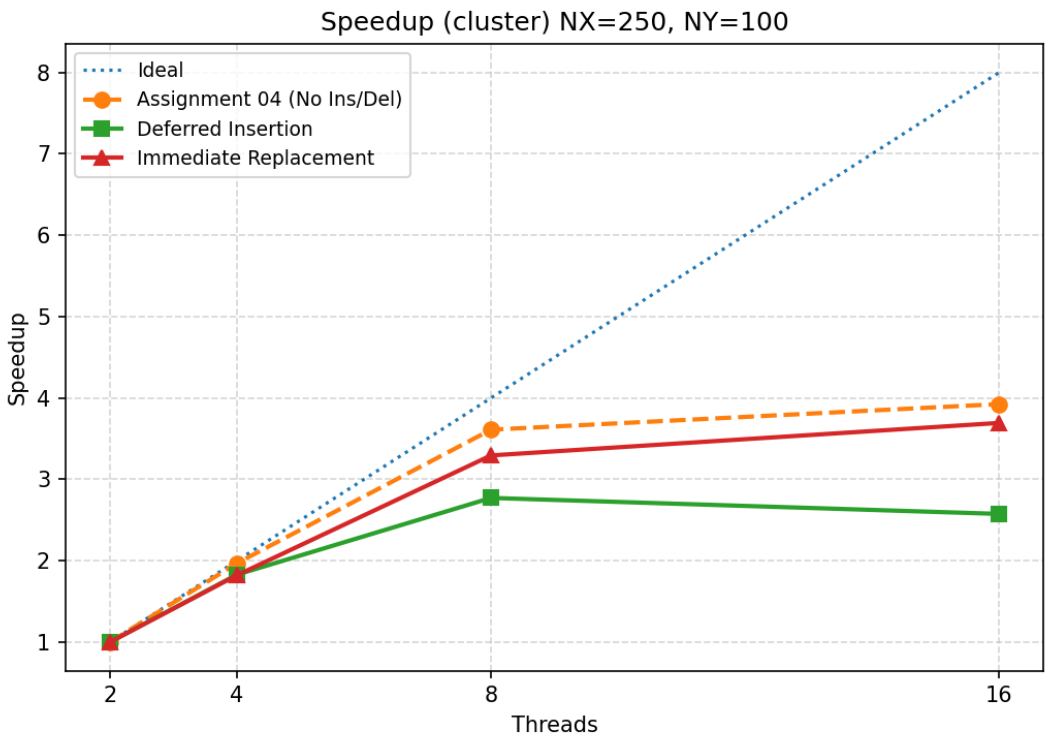

Figure 10: Speedup on HPC Cluster (NX=250, NY=100).

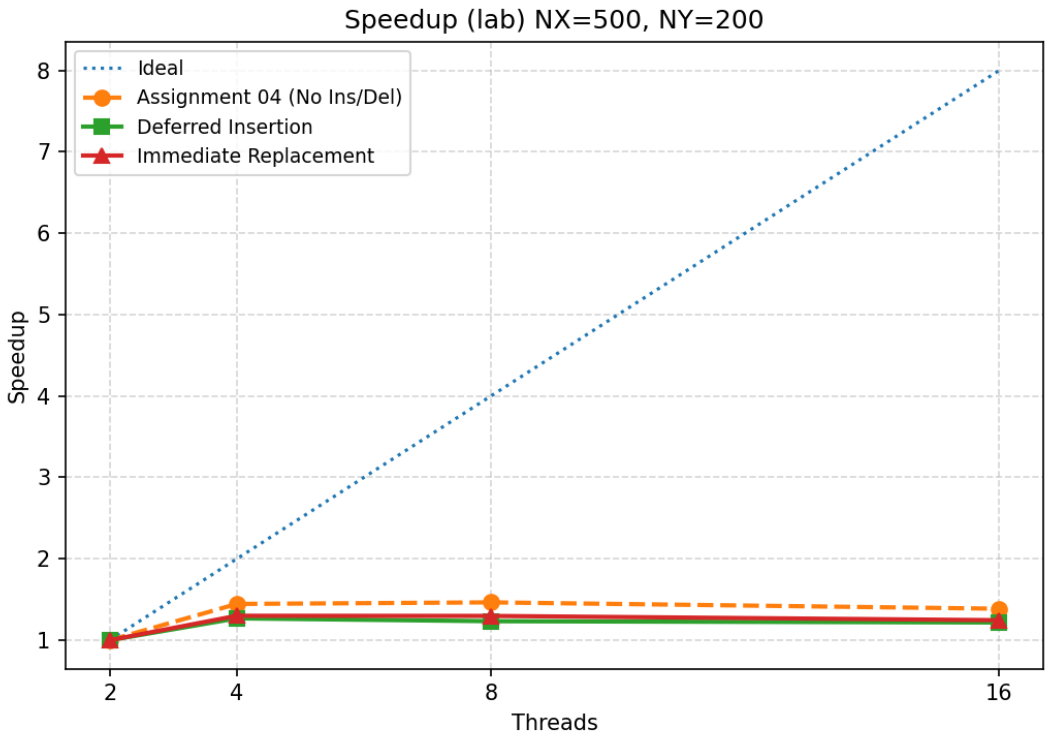

Figure 11: Speedup on Lab PC (NX=500, NY=200).

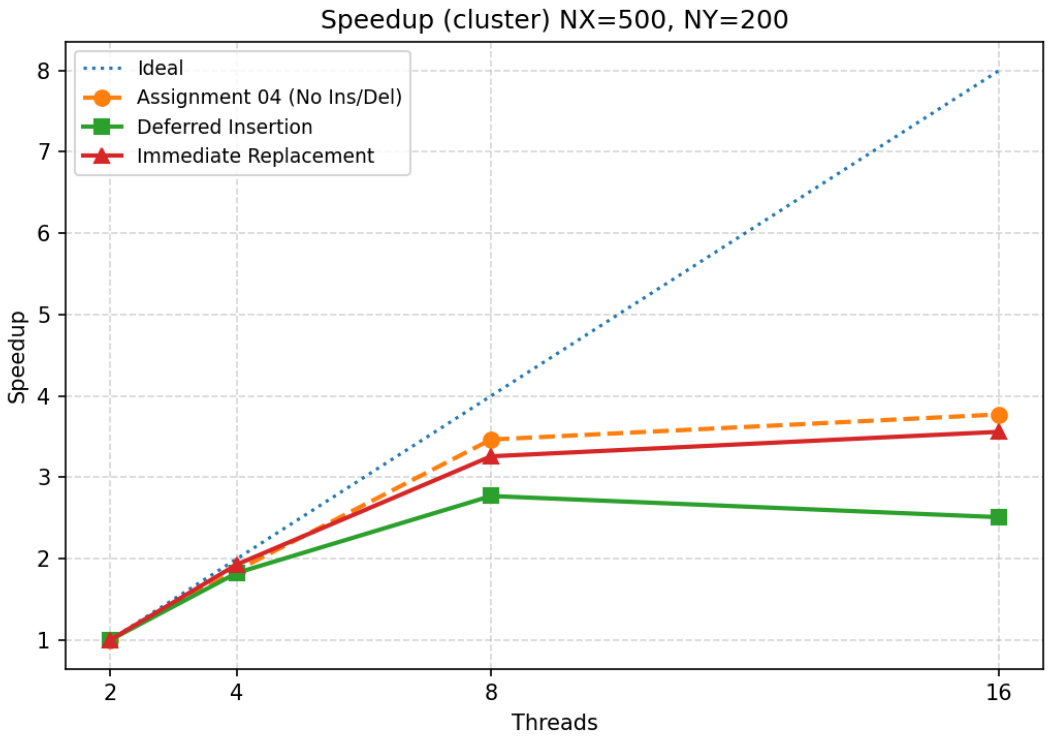

Figure 12: Speedup on HPC Cluster (NX=500, NY=200).

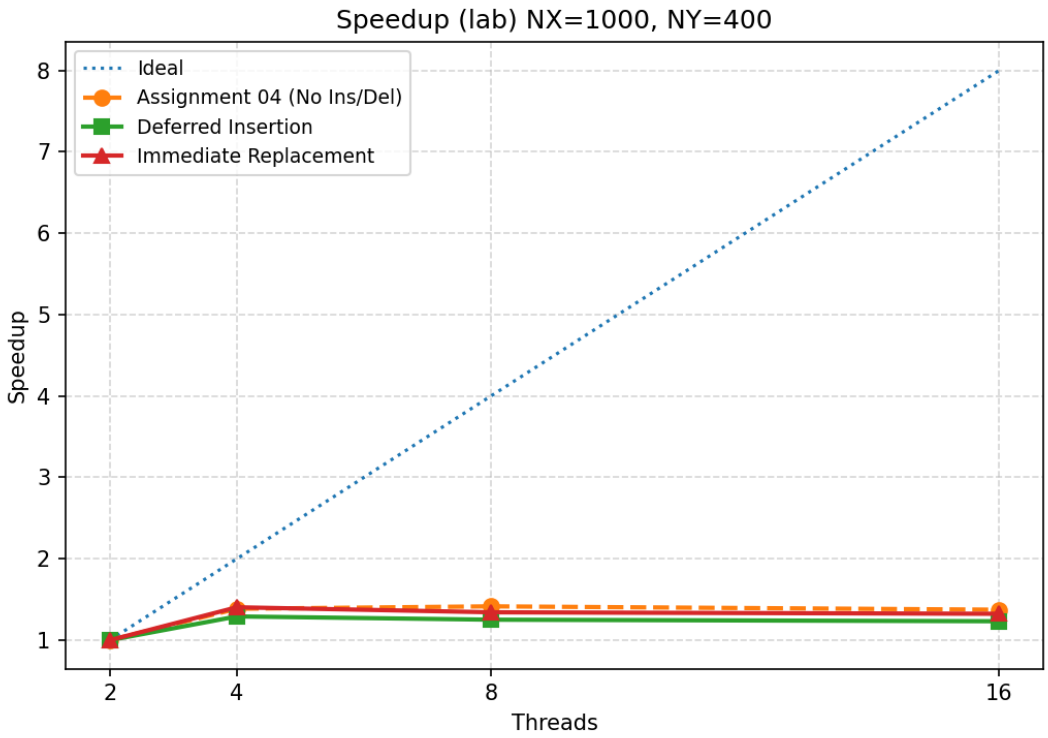

Figure 13: Speedup on Lab PC (NX=1000, NY=400).

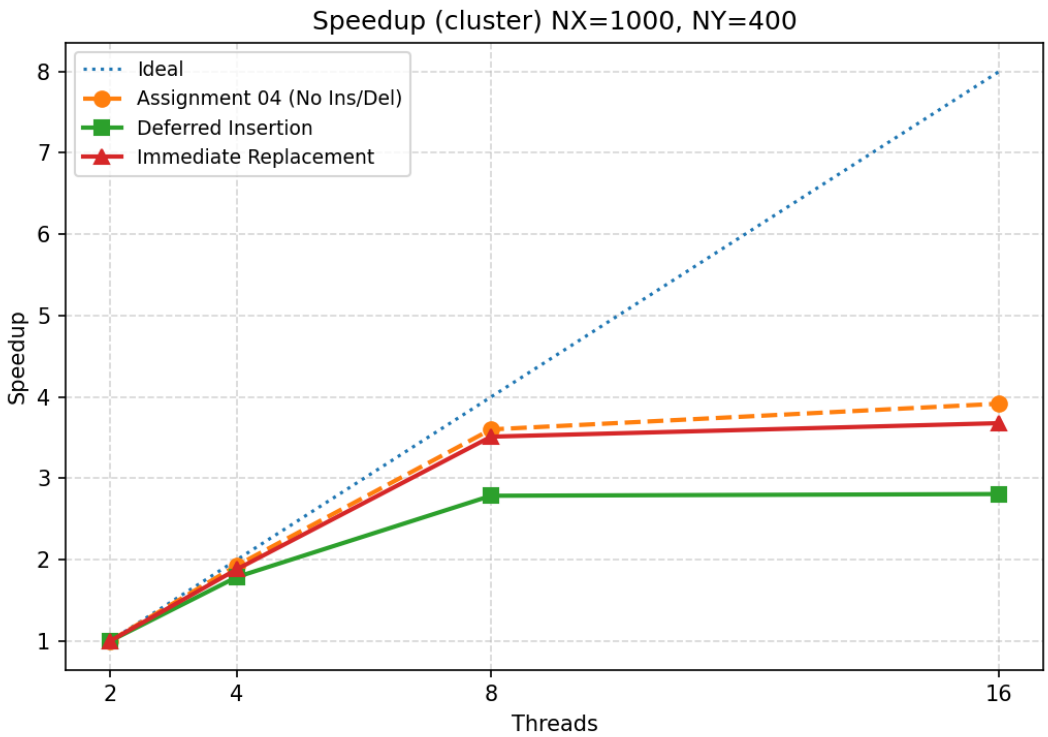

Figure 14: Speedup on HPC Cluster (NX=1000, NY=400).

Observations on Speedup and Scalability:

- Assignment 04 (no ins/del) achieves higher speedup (up to ≈4× on HPC Cluster with 16

threads).

- Both new approaches show reduced speedup (typically 1.2–3.7×), indicating overhead from insertion/deletion operations.
- **Deferred Insertion** generally scales better than Immediate Replacement at higher thread counts because the move phase can be fully parallelized with a single reduction for deletion count.
- Speedup saturates earlier on Lab PC (≈4–8 threads) due to fewer cores and limited memory bandwidth.
- On HPC Cluster, better scaling is observed up to 16 threads thanks to higher core count and superior memory subsystem.

Lab PC vs HPC Cluster: The HPC Cluster consistently shows lower absolute runtimes and better parallel efficiency. This difference arises from architectural advantages: higher number of CPU cores, greater memory bandwidth, improved cache hierarchy, and better support for multi-threaded workloads. Lab PCs, having fewer cores and shared resources, experience earlier saturation and higher thread contention.

5 Pictorial Representation of Approach

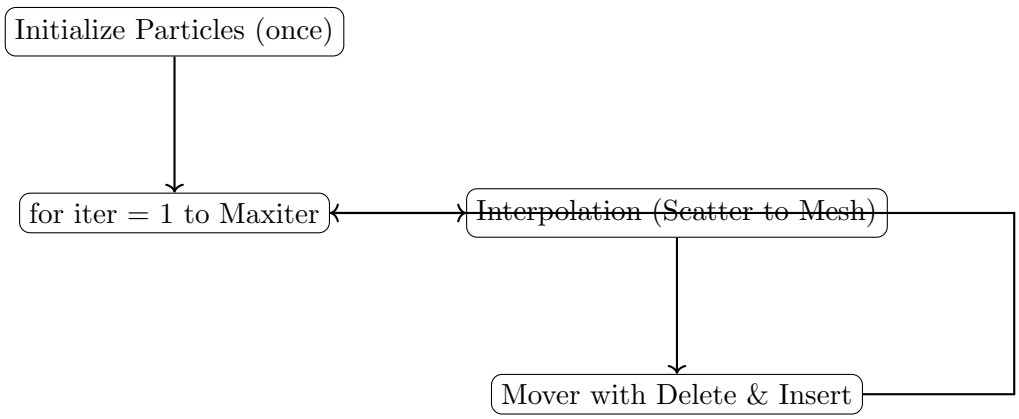

Figure 15: Schematic Flow Diagram of the Simulation Loop.

In the **Deferred Insertion** approach, the Mover consists of a parallel move-and-delete phase followed by a separate insertion phase. In the **Immediate Replacement** approach, replacement occurs inside the parallel loop for each exiting particle.

6 Memory and Computation Analysis

Table 1: Memory Requirement and Analysis (Example: NX=1000, NY=400, ≈14 Million Particles)

Component	**Memory (approx.)**	**Notes**
Particles array (x, y)	224 MB	14M particles × 2 doubles × 8 bytes
Mesh grid	3.2 MB	1000 × 400 doubles
Total	≈ **230 MB**	Heavily dominated by particle data

Memory requirement scales linearly with the number of particles. For large particle counts ($\geq 10^7$), the application becomes increasingly **memory-bound** because of streaming access through large particle arrays with relatively poor temporal locality caused by random displacements. The number of floating-point operations (FLOPs) per particle remains roughly constant (displacements, boundary checks, and random number generation). On modern HPC systems with high memory bandwidth, the bottleneck shifts toward memory access rather than pure computation when particle counts are very high.

7 Detailed Discussion

Impact of Insertions and Deletions on Scalability: The addition of insertion and deletion operations reduces parallel scalability compared to Assignment 04. These operations introduce irregular control flow, extra memory writes, and occasional synchronization points, which limit perfect scaling according to Amdahl's law.

Difficulties Encountered and Measures Taken for Parallel Insertion/Deletion:

- **Thread-safe Random Number Generation**: Solved by implementing per-thread random number generators to avoid contention on a global RNG.
- **Synchronization for deletion counting**: Used OpenMP `reduction` clause in the Deferred Insertion approach.
- **Load imbalance**: Addressed by using `schedule(static)` or `schedule(dynamic)` depending on particle exit patterns.
- **Memory contention during compaction**: Separated the move phase from the insertion phase in the Deferred approach to minimize concurrent writes.

Task Decomposition Performed: The Mover operation was decomposed into: 1. Parallel displacement and boundary checking phase. 2. Deletion handling and counting (via reduction). 3. Insertion of new particles (either deferred in a separate loop or immediate at the same index).

This decomposition allowed the most computationally expensive part (particle movement) to run with high parallelism.

Bonus Question: Different Data Structures Tried Various data structures were experimented with to improve parallel scalability:

- Array of Structures (AoS) – simple and cache-friendly for sequential access but suffered during compaction.
- Structure of Arrays (SoA) – provided better vectorization and cache behavior.
- Linked lists – offered flexibility for insertion/deletion but resulted in poor parallel performance due to irregular memory access.
- Cell-linked particle lists (spatial partitioning) – improved locality but increased synchronization overhead.

The contiguous array combined with the Deferred Insertion approach gave the best overall performance for this assignment.

Bonus Question: Recommended Data Structures for Maximum Parallel Scalability To achieve maximum parallel scalability in future implementations:

- Use **Structure of Arrays (SoA)** layout for particles to enable better vectorization and cache utilization.
- Implement spatial partitioning (e.g., grid-based buckets) to improve data locality.

- Consider lock-free data structures or atomic operations for dynamic insertion/deletion.
- For very large-scale simulations, a hybrid approach combining contiguous arrays with per-cell linked lists can significantly reduce contention while maintaining high parallelism.

8 Conclusion

The parallel Mover with dynamic insertion and deletion was successfully implemented using both Deferred Insertion and Immediate Replacement approaches. Linear scaling with particle count was observed, along with reasonable OpenMP speedup, although lower than the version without insertion/deletion due to added overhead. The HPC Cluster demonstrated significantly better performance and scalability than the Lab PC owing to its superior hardware architecture. The Deferred Insertion approach proved more suitable for parallel execution.

Future work may focus on advanced data structures and NUMA-aware memory allocation to further improve scalability.

All source code is submitted separately as per the assignment guidelines.